



**Universitat Autònoma
de Barcelona**

Informe de progrés II

**Disseny i implementació d'un joc de puzzle tipus
Sudoku incorporant dispositius d'interacció
gestual**

Pablo Chen

Tutor: Enric Marti Godia

**Grau en Enginyeria Informàtica
Escola d'Enginyeria
Curs 2021-22**

Control del document

Informació del document

	Informació
Identificació del document	<i>Informe Inicial</i>
Autor[es] del document	<i>Pablo Chen</i>
Data de creació	<i>14/02/2022</i>
Data del document	<i>28/02/2022</i>
Nombre del fitxer	<i>Informe-v3.pdf</i>

Historial del document

Versió	Data versió	Canvis
<i>1.0</i>	<i>14/02/2022</i>	<i>Introducció Paraules Clau Motivació Objectius del Projecte Planificació</i>
<i>2.0</i>	<i>10/04/2022</i>	<i>Informe de progrés I</i>
<i>3.0</i>	<i>22/05/2022</i>	<i>Informe de progrés II</i>

Aprovació del document

Perfil	Nom	Firma	Data
Alumne	Pablo Chen		14/02/2021
Tutor	Enric Marti Godia		14/02/2021

Taula de continguts

Disseny i implementació d'un joc de puzzle tipus Sudoku per a aprendre els números en Braille incorporant dispositius d'interacció gestual.....	1
1. Resum.....	4
2. Paraules clau.....	4
3. Motivació.....	4
4. Objectius del projecte.....	4
5. Planificació.....	5
6. Metodologia.....	6
7. Desenvolupament del projecte.....	6
7.1. Estat de l'art.....	6
7.2. Disseny de les classes principals.....	6
7.3. Disseny de la interfície del sudoku.....	6
7.4. Desenvolupament de les classes principals del joc.....	6
7.4.1. Part 1.....	6
7.4.2. Part 2.....	7
7.5. Desenvolupament del joc.....	9
7.6. Testejar els diferents components del joc.....	10
Bibliografia y referències.....	12

1. Resum

En aquesta secció es farà un resum del projecte: necessitats que pretén cobrir, de què va, objectiu principal.

2. Paraules clau

Videojoc, Sudoku, Braille, OpenGL, Gràfics per Computador, Motor de joc.

3. Motivació

El sudoku és un joc que consisteix a emplenar una quadrícula de 9 x 9 amb números de l'1 al 9 de manera que no es repeteixi cap número en cada fila, columna o subquadrícules 3 x 3 que componen el tauler. En l'actualitat, podem trobar sudokus en moltes parts: en els diaris, en llibres que recopilen sudokus per resoldre, en aplicacions per al mòbil o ordinador i fins sudokus amb peces en braille. Tot i això, no s'ha pogut trobar un sudoku 3D en braille per a ordinador que permeti aprendre els números en braille. És per això que s'ha proposat a realitzar aquest TFG. A més a més, estic interessat a aprendre a desenvolupar jocs amb OpenGL.

4. Objectius del projecte

El principal objectiu del projecte és el disseny, implementació i test d'un joc 3D, concretament, un sudoku 3x3 amb els números en braille. El joc haurà d'implementar les següents funcionalitats:

- Guardar i carregar partides a partir d'un fitxer de text.
- Generar un sudoku aleatori d'una determinada dificultat.
- El sudoku podrà donar pistes quan el ratolí estigui a sobre d'una casella.
- Es podrà solucionar el sudoku de manera interactiva, pas a pas.
- Es podrà la modalitat del sudoku: d'un sudoku clàssic a un sudoku mini o una altra varietat.
- El joc ha de poder canviar la numeració de la interfície (de braille a aràbiga).
- El joc ha de poder ser compilat i executat en diferents plataformes (Linux i Windows).

El joc haurà de gestionar els següents esdeveniments:

- Un clic i doble clic.
- Pulsacions de tecles.
- Moviments del ratolí.
- Altres perifèrics com ara Kinect.
- Esdeveniments de la finestra (canvi de mida).

Per poder implementar les diferents funcionalitats del joc, es desenvoluparà varies classes per gestionar la renderització dels objectes 3D i la interacció amb aquests. El

joc haurà de renderitzar objectes simples com ara: un cub, una esfera o un cilindre a partir de primitives simples, a més, ha de permetre traslladar, girar i escalar els objectes 3D. Així mateix, s'haurà d'implementar una càmera i poder moure aquesta mitjançant diferents perifèrics.

Respecte a les eines i llibreries de desenvolupament, s'utilitzara principalment, l'entorn de desenvolupament multiplataforma Clion per C/C++ per poder desenvolupar el joc tant en Windows com en Linux i SHADERed, un debugger per als shaders d'OpenGL. Respecte a les llibreries, es farà servir ImGui, per generar la GUI, GLFW per i els perifèrics, OpenGL per interactuar amb la GPU, la llibreria GLAD per carregar les extensions d'OpenGL i GLM per realitzar càlculs amb matrius i vectors.

5. Planificació

El projecte es divideix en 7 tasques, que incluen dues parts que comprenen la major part del treball: el disseny i el desenvolupament.

Tasca	Descripció	Durada (setmanes/hores)	Grau Finalització
Estat de l'Art	Buscar productes/aplicacions similars al que es proposa, caracteritzar-los i comparar-los	5 h	0 %
Disseny de les classes principals	Dissenyar les classes de renderització i els altres components del joc.	15 h	90 %
Disseny de la interfície del sudoku	Dissenyar les diferents pantalles del joc: la pantalla de inici, els controls del sudoku, etc.	15 h	70 %
Desevolupament de les classes principals del joc	Implementar el la classe de renderització amb OpenGL, implementar les rutines de tractament de les accions dels perifèrics	150 h	90 %
Desenvolupament del joc	Implementar la lògica i les funcionalitats principals del joc.	50 h	50 %
Testejar els diferents components del joc	Tests per a cada component implementat.	30 h	20 %
Memòria i Presentació	Escriure la memòria i la presentació oral	10 h	0 %
GRAU DE MADURESA:			71 %

Taula 1. Planificació de tasques del projecte.

6. Metodologia

S'utilitzarà el programari git per fer un control de versió del codi font del joc i Github per hostejar el codi font. Se seguirà una metodologia àgil, concretament, s'utilitzarà una tècnica anomenada Kanban, es dividiran les tasques en diferents subtasques i es col·locarà cada tasca segons el progrés (*todo, in progress and done*). En particular, és fara servir KanbanFlow per gestionar les subtasques.

7. Desenvolupament del projecte

En els següents apartats s'explicarà el desenvolupament dels diferents mòduls que conformen el projecte.

7.1. Estat de l'art

7.2. Disseny de les classes principals

7.3. Disseny de la interfície del sudoku

El joc haurà de tenir com a mínim 4 botons per completar els objectius.

- Un boto per guardar la partida.
- Un boto per carregar la partida.
- Un boto per canviar de braïlle a numèric.
- Un boto per activar el solucionador interactiu. Aquesta constarà de dues fleches que permetrà al jugador avançar i retrocedir en la solució.

Tots aquets elements tindran tecles d'accés ràpid per millorar la comoditat i l'accessibilitat.

7.4. Desevolupament de les classes principals del joc

7.4.1. Part 1

Un joc, normalment, està format per tres grans components (motor de renderització, motor de física i una llibreria matemàtica) que interactuen entre si per tal de permetre la interacció amb les entitats del joc. En el nostre cas, degut a que no implementarem un joc molt complex, només implementarem les funcionalitats que facin falta, es a dir un sistema per renderitzar objectes, un sistema d'esdeveniments per gestionar la E/S de l'usuari i si fa falta un sistema de col·lisions.

Primer de tot, s'ha creat unes funcions que s'encarreguen de la inicialització i destrucció del context OpenGL i de la finestra per on renderitzarem els nostres resultats. Seguidament, s'ha definit unes classes (*VertexBuffer*, *IndexBuffer* i *VertexArrayBuffer*) que abstrueixen els diferents buffers d'OpenGL per tal de poder més còmodament, aquesta classe té una rutina molt important (*draw*) que serveix per pintar l'objecte en pantalla i es crida en cada refresc de la pantalla.

Per una altra banda, s'ha implementat una classe *Shader* que s'encarrega de comprovar si hi ha errors en el shader i de inicialitzar el programa shader. La classe, també implementa mètodes per assignar les diferents variables *uniform* (int, float, vec3, mat4, etc).

En quant al sistema d'esdeveniments, s'ha creat una classe anomenada *Event* la qual es heretada per els diferents tipus d'events (*MouseMoveEvent*, *MouseButtonEvent*, *KeyEvent*, *WindowCloseEvent* i *WindowResizeEvent*), també s'ha implementat una funció que instal·la els *callbacks* al sistema d'events del GLFW.

Una vegada desenvolupat el sistema d'esdeveniments i les classes bàsiques per renderitzar, es va implementar una classe *Camera* que controla el la visualització del joc.

La classe *Camera* incorpora una funció que permet actualitzar l'estat apartir dels events del teclat i el ratolí. Finalment, es va implementar la classe *Object* que permet renderitzar un cub amb les transformacions que vulguis.

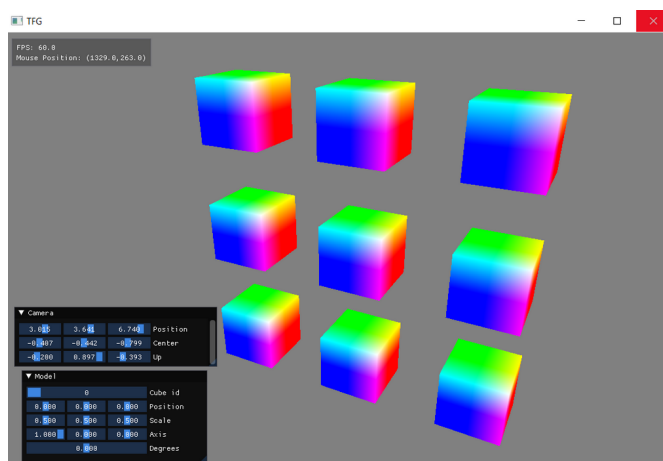


Figura 1: Prova de concepte de la classe de renderització del cub.

7.4.2. Part 2

En aquesta segona part, es va millorar algunes classes i es va acabar de implementar les funcionalitats que es necessitaven pel joc:

- Es va substituir les classes *VertexBuffer*, *IndexBuffer* i *VertexArrayBuffer* per un mètode virtual en *Object* que s'encarrega de inicialitzar els buffers degut aquestes només s'utilitzavem la classe *Object* i eren redundants.
- Es va crear la classe estàtica *Injector* per proveir les dependències a les classes que ho necessitin i així eliminar els paràmetres dels constructors, també conté utilitats tal com
 - *ScreenToWorld*: Mapeja les coordenades del ratolí a coordenades 3D.
 - *ScreenToColor*: Obté el color de la coordenada del ratolí, s'utilitza per implementar el *Picking Object*.
 - *ColorToId*: Transforma el color RGB al identificador del objecte.

- GenPickColor: Codifica un identificador en un color.
- Es va implementar classe *Sphere* per renderitzar una esfera de radi R amb amb N sectors i T talls.
- Es va crear la estructura de dades *Transform* per tal de separar les transformacions geometriques de les classes de renderització.
- S'ha implementat un punt de mira utilitzant la classe *Object*. El punt de mira servira perque el jugador pugui tenir localitzat el ratolí en tot moment.

```
float r = int((n % 10)) / 10.f;
float g = int((n % 100) / 10) / 10.f;
float b = int((n % 1000) / 100) / 100.f;
```

Figura 2: Fòrmula de codificació.

```
int id = std::round(color.r * 10) +
        std::round(color.g * 100) +
        std::round(color.b * 1000);
```

Figura 3: Fòrmula de decodificació.

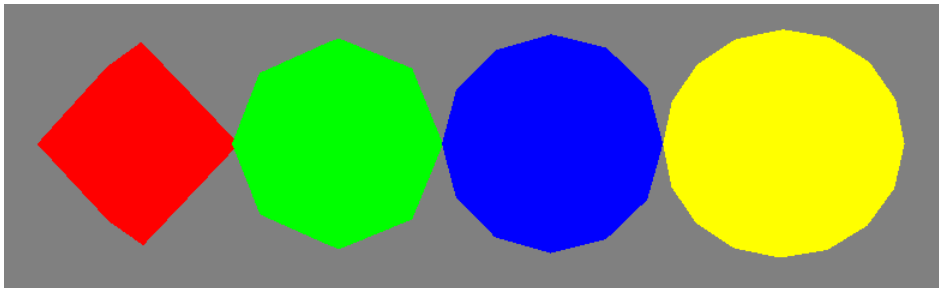


Figura 4: Renderització d'esferes amb diferents paràmetres.

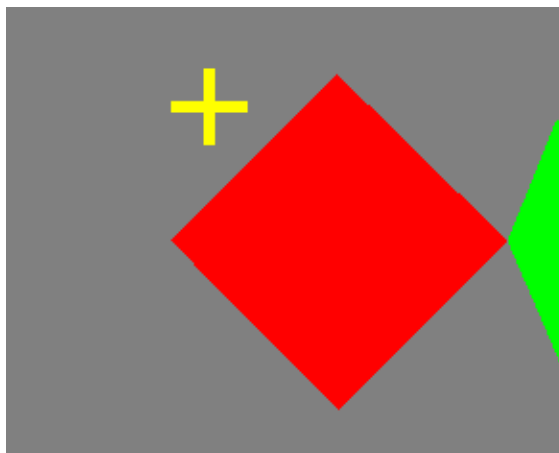


Figura 5: Punt de mira amb amb escalat 20 vegades.

7.5. Desenvolupament del joc

Per implementar el joc, es va desenvolupar dues classes:

- La classe *Board* que serveix per gestionar la posició dels cubs i les esferes del món 3D. La classe pot crear fitxes de diferents valors en posicions arbitràries del món 3D.
- La classe *Sudoku* que conté la lògica del joc inicialitza la classe *Board*. El solucionador, la generació de noves partides i el guardat s'implementara en aquesta classe.

La posició de les esferes en la fitxa es va implementar mitjançant una *lookup table*; una llista on el valor numèric indica l'index de les esferes que s'han de dibuixar.

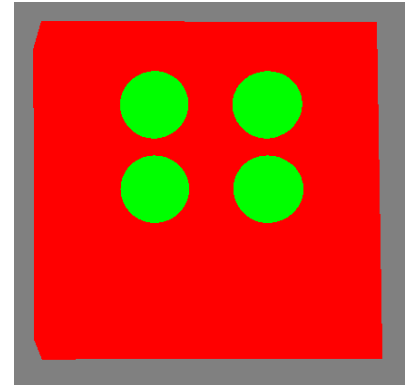


Figura 6: Fitxa amb el numero 7 en braille.

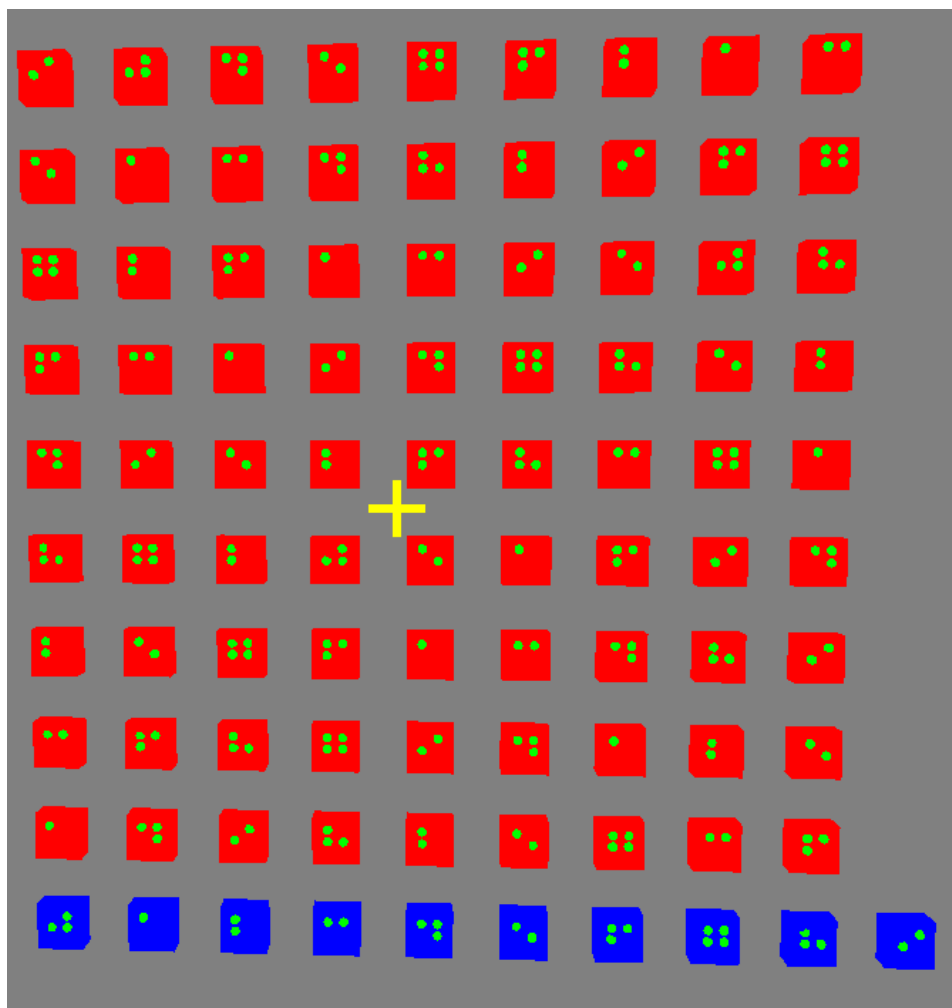
```
number2Braille_ = {  
    {1, 2, 3}, // 0  
    {0}, // 1  
    {0, 2}, // 2  
    {0, 1}, // 3  
    {0, 1, 3}, // 4  
    {0, 3}, // 5  
    {0, 1, 2}, // 6  
    {0, 1, 2, 3}, // 7  
    {0, 2, 3}, // 8  
    {1, 2}, // 9  
    {0, 1, 2, 3, 4, 5} // 10 draw everything  
};
```

Figura 7: Taula utilitzada per renderitzar les esferes.

7.6. Testejar els diferents components del joc

TODO:

- mode numeric
- *guardar partides*
- solucionador interactiu.
- uml
- implementar interfície
- test



Bibliografia y referències

[OpenGL] www.opengl.org web oficial de la llibreria OpenGL amb informació sobre extensions de la llibreria i FAQ's (data últim accés: febrer 2022)

[OpenGL docs] docs.glfw.org OpenGL API Documentation (data últim accés: febrer 2022)

[KanbanFlow] kanbanflow.com Project Management Kanbanflow (data últim accés: març 2022)

[Sudoku] sudoku.com Sudoku online. (data últim accés: març 2022)

[MaxiAids] maxiaids.com/braille-sudoku-puzzle-game-with-board Braille Sudoku Puzzle Game. (data últim accés: març 2022)

[GLAD] <https://github.com/Dav1dde/glad/tree/c> OpenGL Loader

[ImGui] <https://github.com/ocornut/imgui> Graphical User interface for C++

[GLM] <https://github.com/g-truc/glm> OpenGL Mathematics (GLM)

[Render Doc] <https://renderdoc.org/> Graphics debugger